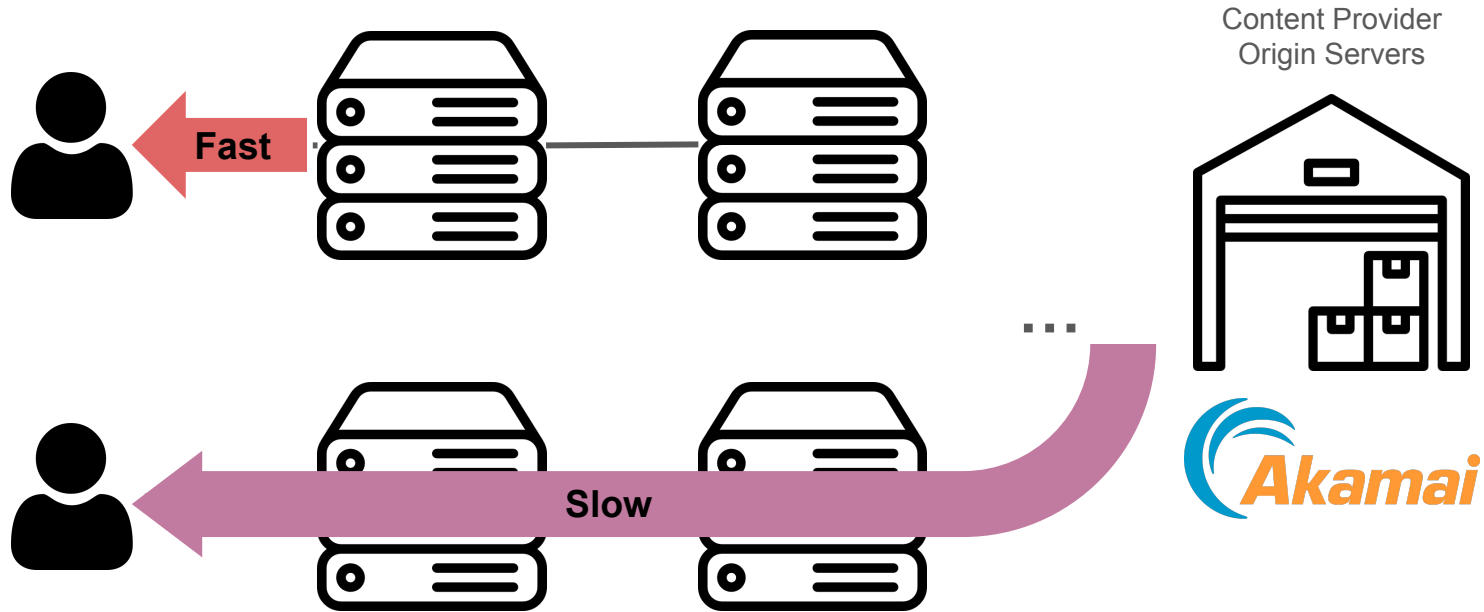


Darwin: Flexible Learning-based CDN Caching

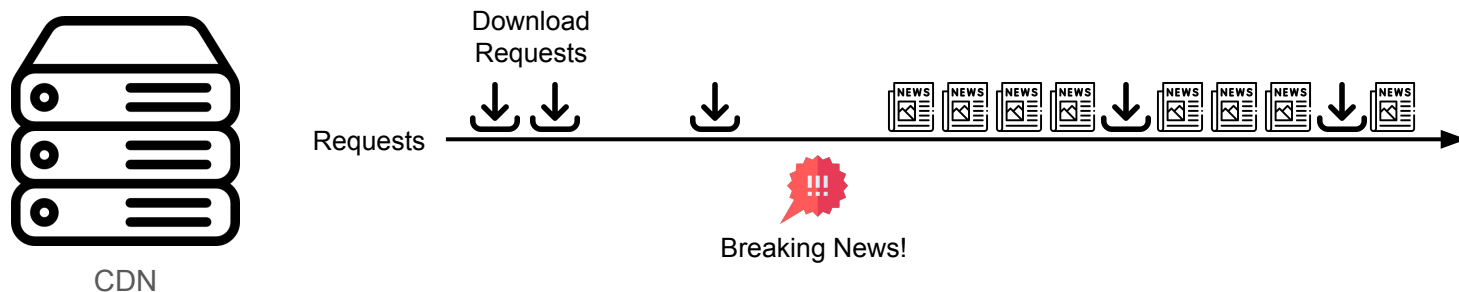
Jiayi Chen, Nihal Sharma, Tarannum Khan, Shu Liu, Brian Chang, Aditya Akella,
Sanjay Shakkottai, Ramesh K. Sitaraman

Presented by: Joon Ha Kim

Content Delivery Networks (CDN)

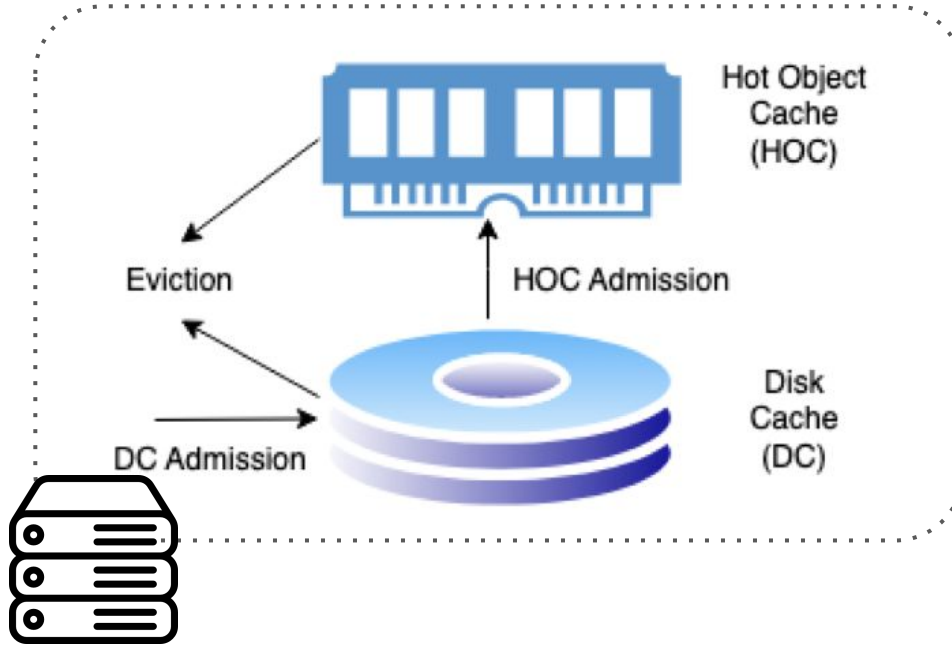


Traffic Changes Make Cache Management Challenging



Can ***static*** cache management policies deal with ***change in workload pattern?***

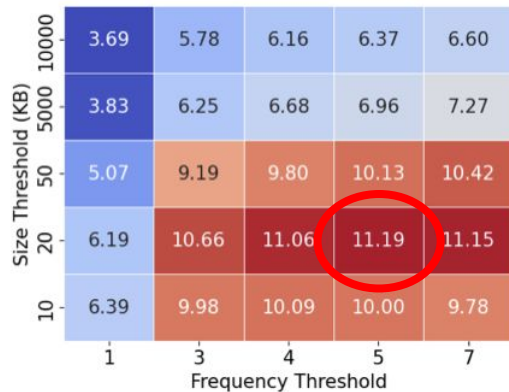
Hot Object Cache (HOC) Admission Policy



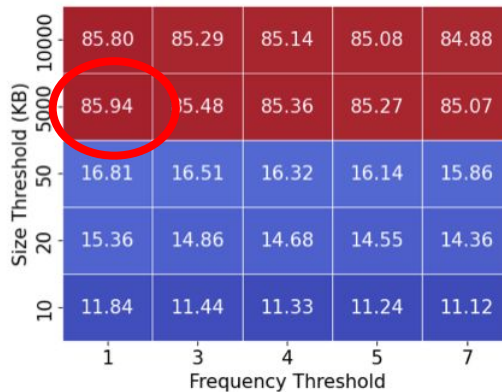
Common Policy:

- $Frequency \geq f$
- $Size \leq s$

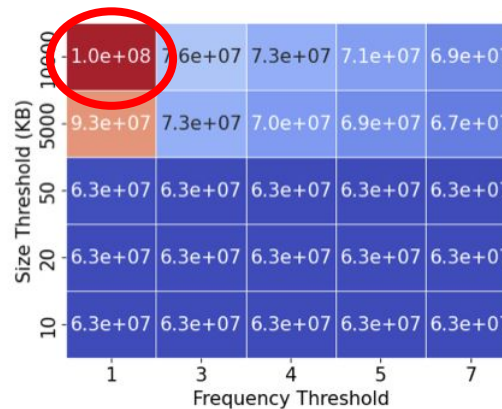
Limitations of Static HOC Admission Policies



(c) Image OHR

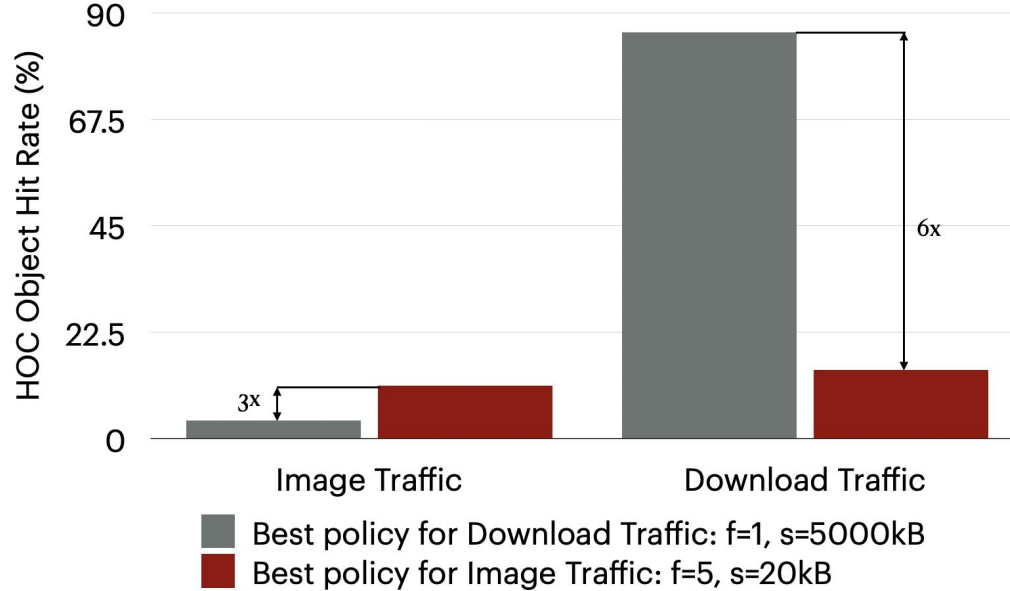


(d) Download OHR



(e) Download Disk Write

Limitations of Static HOC Admission Policies



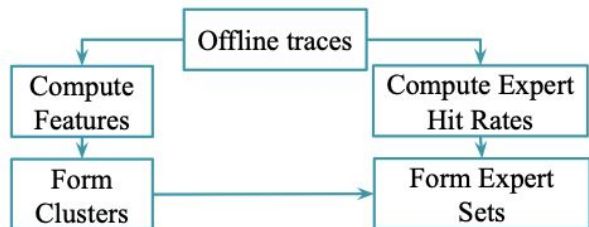
Existing Work and Limitations

Name	Year	Many Knobs	Diverse Goals	Low Overhead	Theoretical Guarantees
LHR [49]	2021	✓	✗	✗	✗
RL-Cache [22]	2019	✓	✓	✗	✗
AdaptSize [10]	2017	✗	✗	✓	✗
Hill Climbing [10]	2017	✓	✓	✗	✗
Percentile [10]	2017	✓	✓	✓	✗

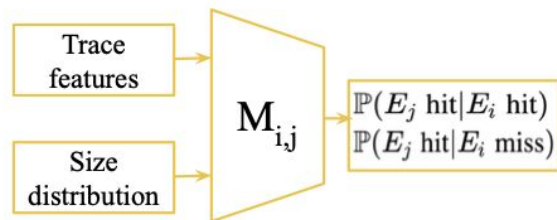
- *AdaptSize@NSDI'17*: **Restrict policy decision knobs** (size)
- *HillClimbing@NSDI'17*: **Don't accommodate other** metrics (disk operations)
- *RL-Cache@NetAI'19*: High **overhead** (per-request inference)

Darwin Overview (Offline + Online)

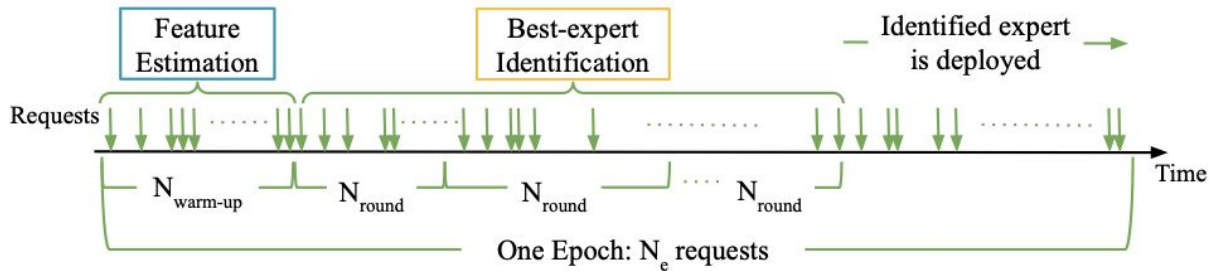
Step 1a. Offline Clustering and Expert Sets



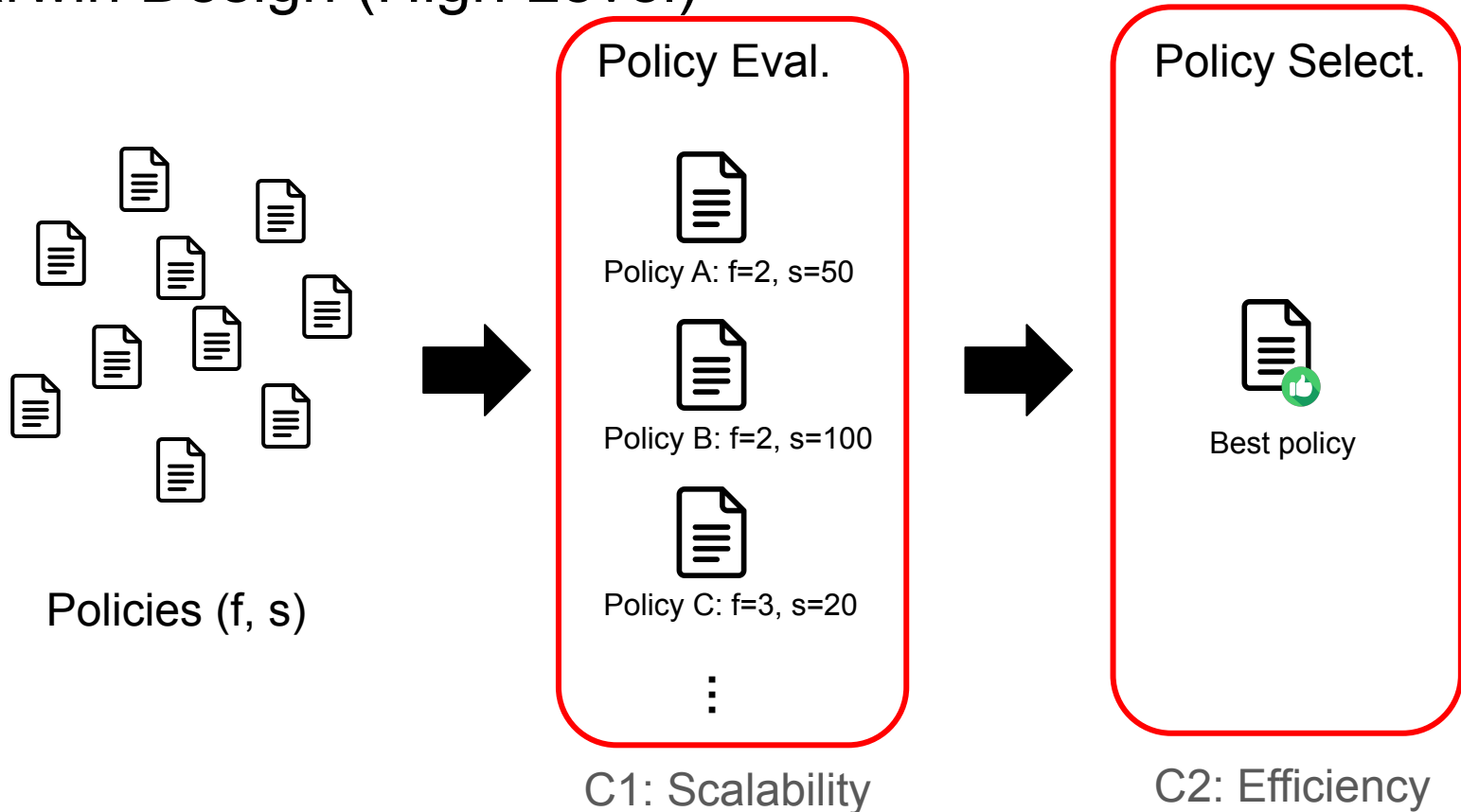
Step 1b. Cross-Expert Prediction Networks



Step 2. Online Selection



Darwin Design (High Level)



Challenge 1: Scalable Performance Observation

Policy Eval.



Policy A: $f=2$, $s=50$



Policy B: $f=2$, $s=100$



Policy C: $f=3$, $s=20$

⋮

Sequential Approach → Too many observation rounds ✖

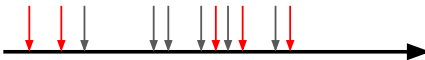
Parallel Approach → Requires too much resource overhead ✖

Insight: Policies overlap! How can we leverage this...?

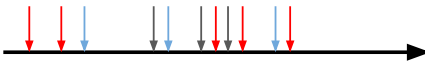
Policy A: $f=2$, $s=50$



Requests



Requests

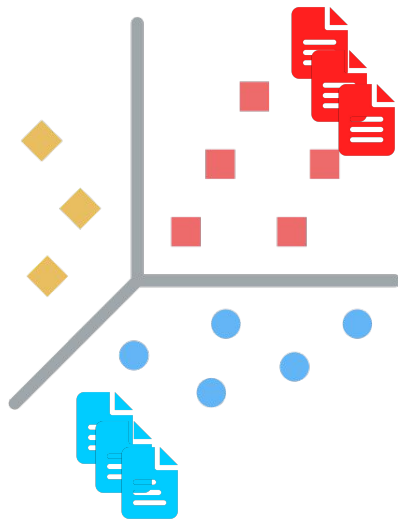


Policy B: $f=2$, $s=100$

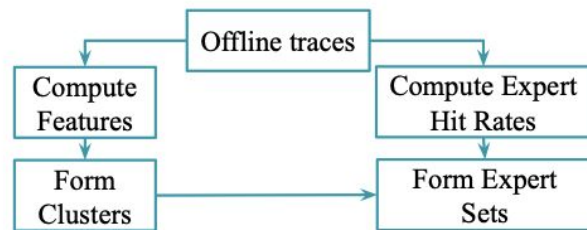
A lot of overlap between requests that **will be promoted** and those that **won't**

Design Solution: Cross Policy Prediction Model

Step 1a: **Feature** Clustering and **Expert Set** Formation



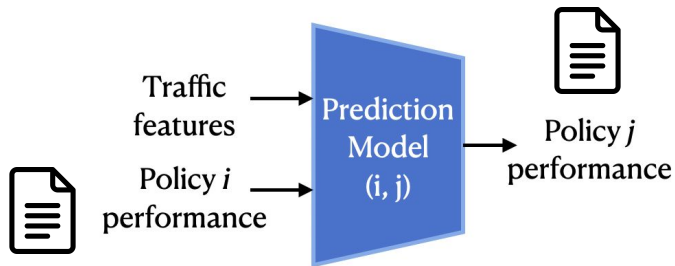
Step 1a. Offline Clustering and Expert Sets



Design Solution: Cross Policy Prediction Model

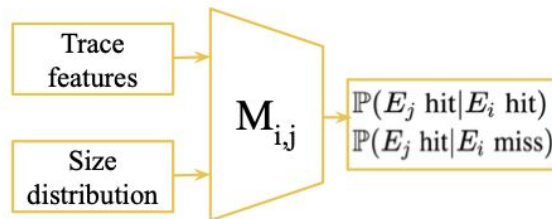
Step 1a: **Feature** Clustering and **Expert Set** Formation

Step 1b: Predicting Expert Policy j given Expert Policy i performance



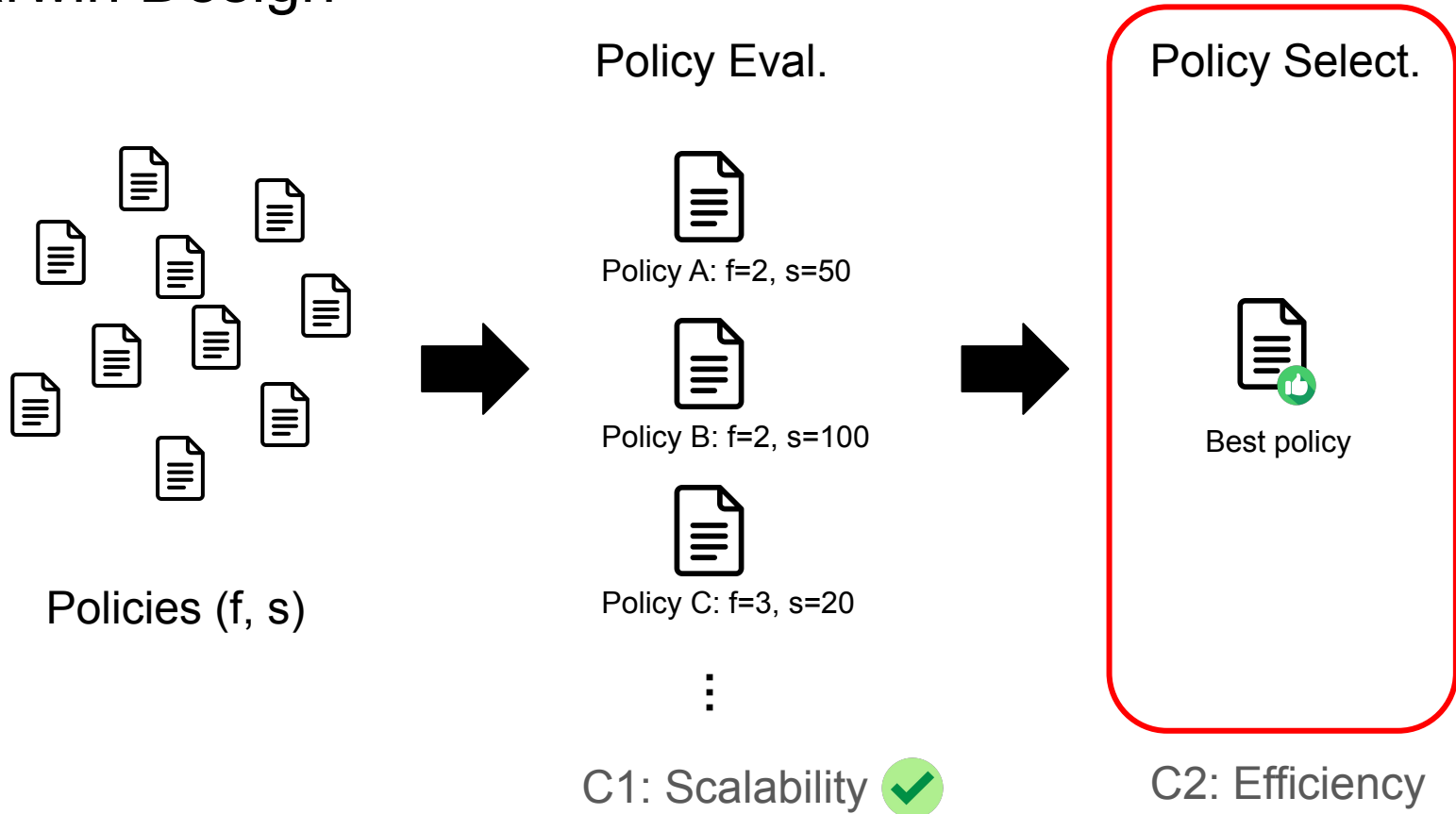
1 layer fully connected
neural network

Step 1b. Cross-Expert Prediction Networks



Note: Darwin uses 36 experts. They have a model for every pair ($36 * 35 = 1260$ total prediction models)

Darwin Design



Challenge 2: Efficient Policy Selection

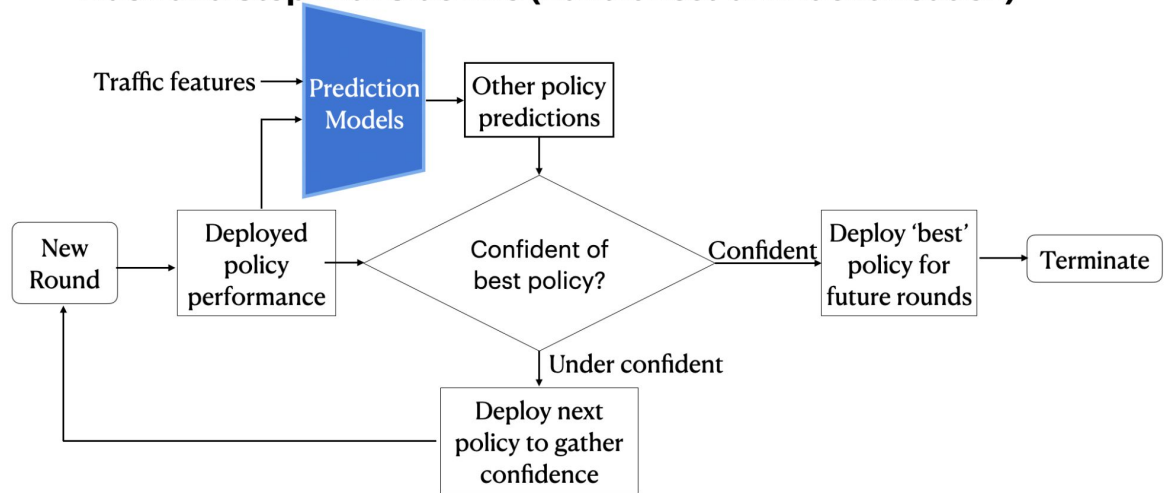
Policy Select.



Best policy

C2: Efficiency

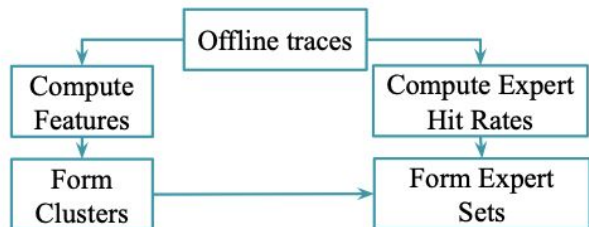
Track and Stop with Side Info (Bandit Best-arm Identification)



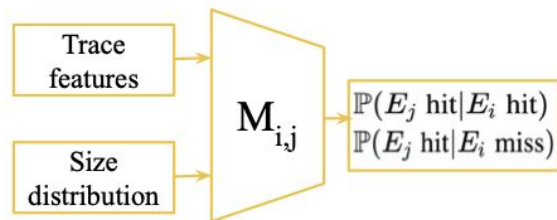
Convergence time is bounded by a constant that is independent of the number of policies.

Darwin Overview (Offline + Online)

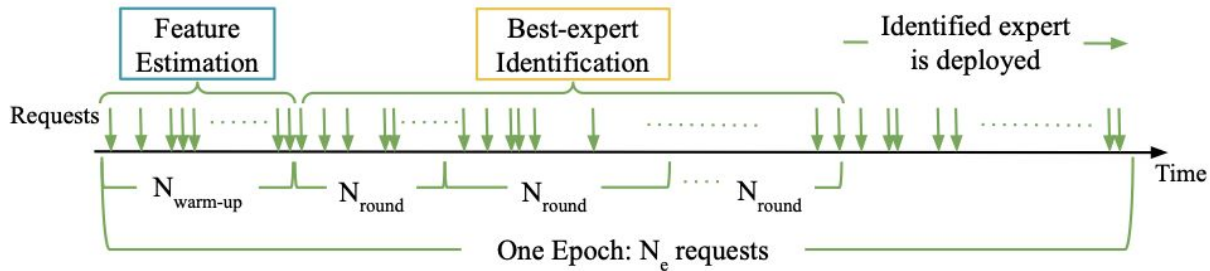
Step 1a. Offline Clustering and Expert Sets



Step 1b. Cross-Expert Prediction Networks



Step 2. Online Selection

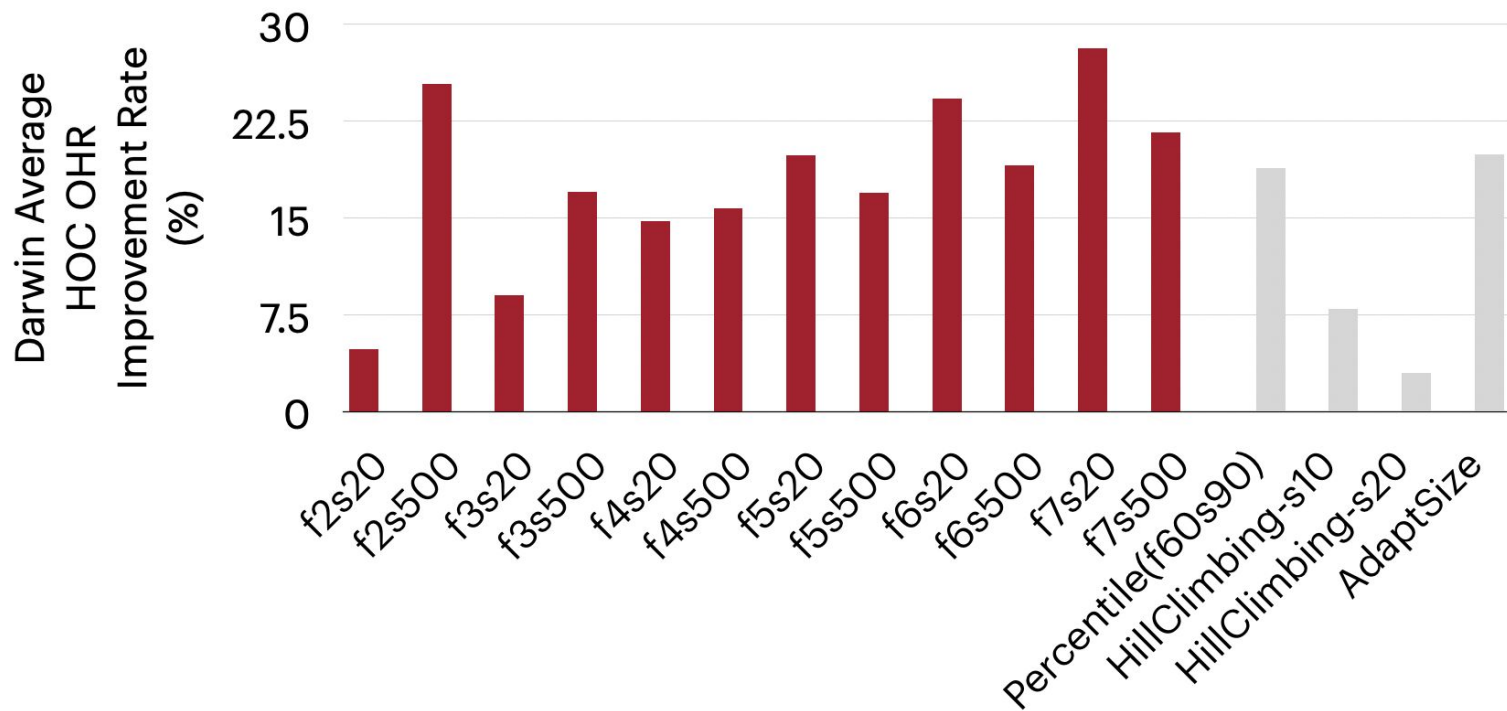


Evaluation Setup

- HOC Cache Size of 100MB, 200MB, 500MB
- CDN Traces (100 Mixed Configs for two traffic classes)
- Baselines (AdaptSize, HillClimbing, Percentile)

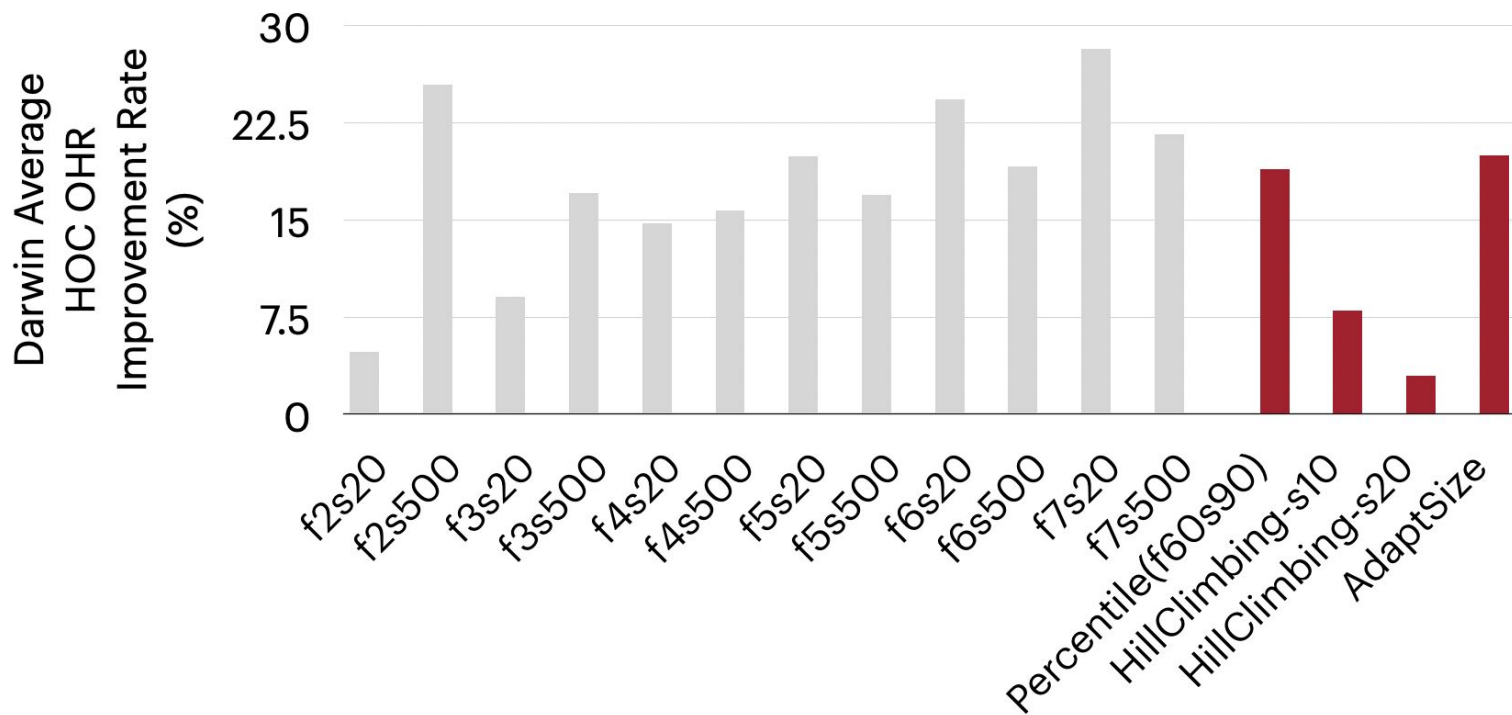
Robustness to Traffic Changes

Darwin outperforms static baselines by 4.83%-28.16%



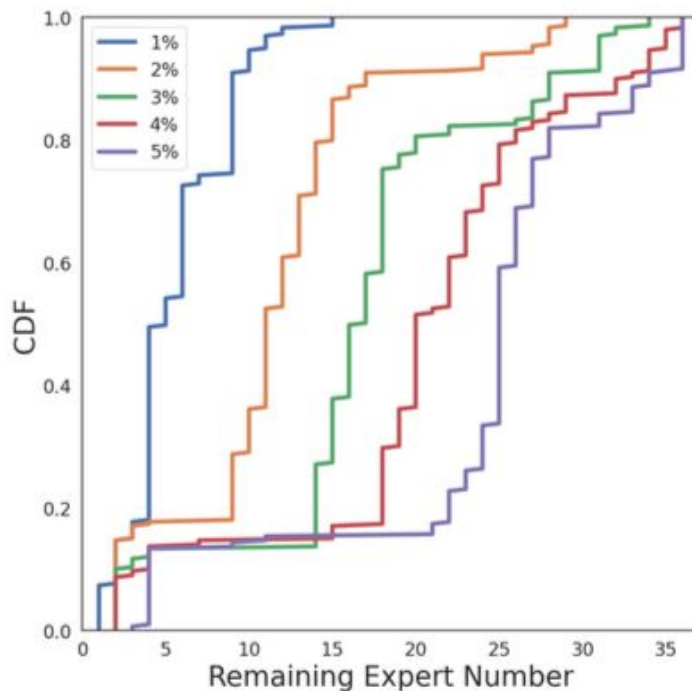
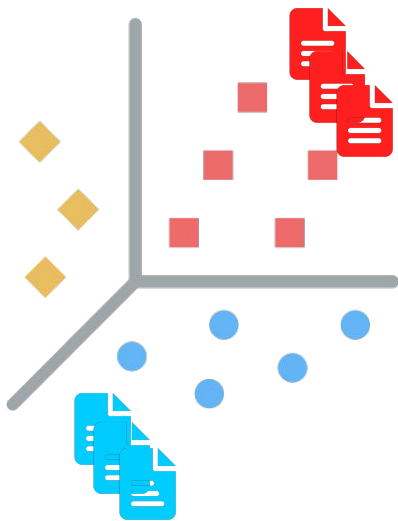
Robustness to Traffic Changes

Darwin outperforms adaptive baselines by 3%-19.96%



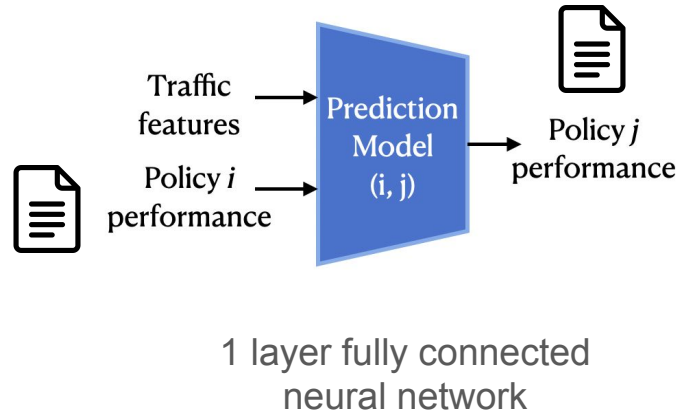
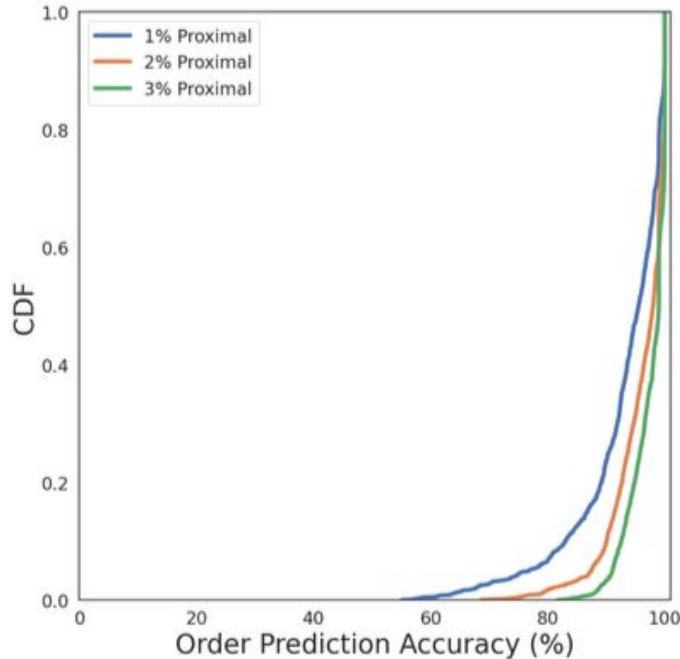
Evaluation: Cross Policy Prediction Model

Step 1a: **Feature** Clustering and **Expert Set** Formation



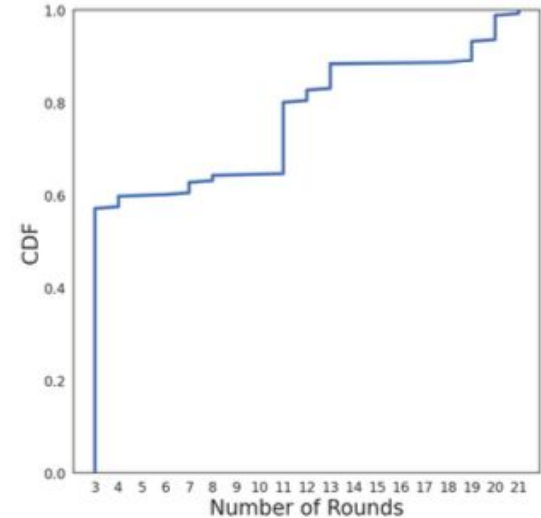
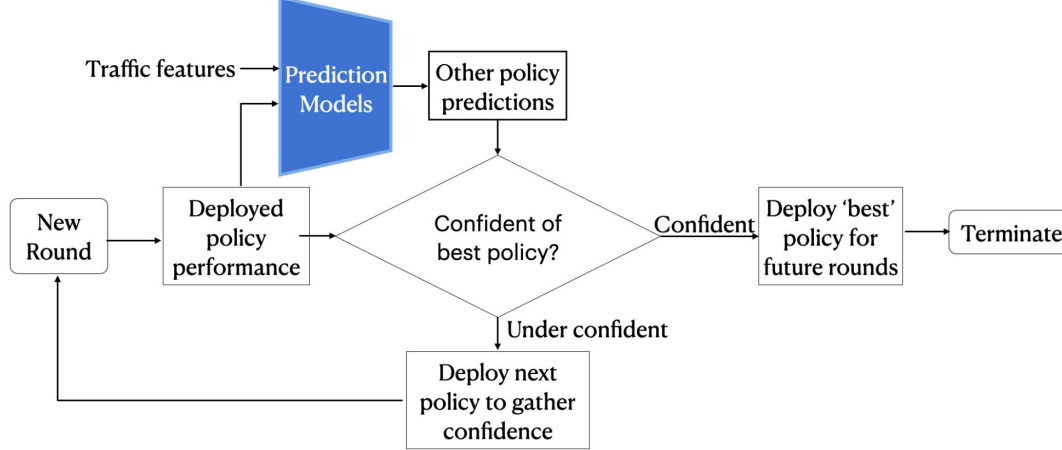
Evaluation: Cross Policy Prediction Model

Step 2: Predicting Policy j given Policy i performance



Evaluation: Bandit Algorithm Convergence

Track and Stop with Side Info (Bandit Best-arm Identification)



(d) Bandit Rounds for Identification

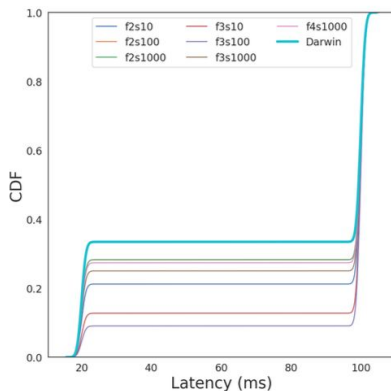
100 Million Request Traces

Takes **first 5.5%** to converge, 80% of the time

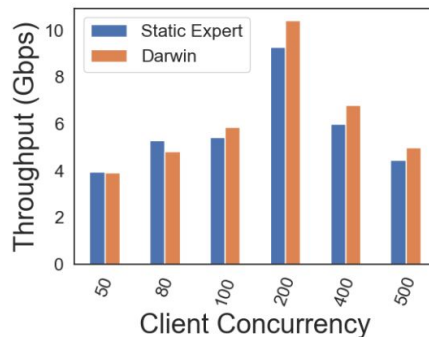
Takes **first 10%** to converge, 10% of the time

Other Metrics

- Darwin can be used to improve other metrics
- Darwin does not impose additional latency overhead
- Darwin minimally impacts CPU and Memory Utilization



(a) First Byte Latency



(b) Throughput

Figure 7: Darwin prototype's latency and throughput performance

Further Discussion

- How challenging is it to modify Darwin for other metrics/goals?
 - e.g., What changes would be needed in the offline / online phases?
- What assumptions is the paper making regarding their ability to support other metrics/goals?
 - e.g., How can we formulate the reward function for the Bandit Algorithm?
 - Seems like this is still heuristic based unless the goal is clear (Hit Ratio)

Thank you.